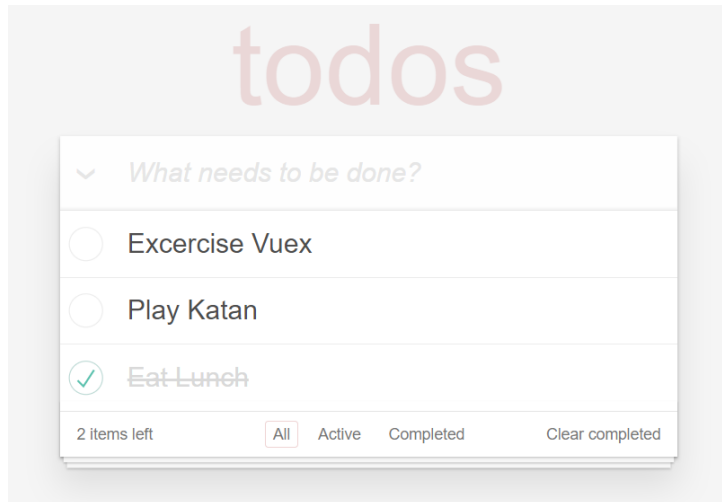


Todo, Todo



Build a [todo-app](#) application that uses a store to manage the state

Our data model:

```
const todo = {  
  _id: "gZ6Nvy",  
  txt: "Master Redux",  
  importance: 9,  
  isDone: false,  
  createdAt: 1711472269690,  
  updatedAt: 1711472269690  
}
```

You will start from a starter project with local component state and modify it to work with state management.

Comment

The starter project was crafted by taking the car-project and perform a search-and-replace (Ctrl+Shift+H with match case) in the entire project of:

- car to todo
- maxSpeed to importance
- vendor to txt
- Speed to Importance
- .id to ._id
- Being careful to avoid renaming in library files
- Changed bunch of file names
- From the UserAuth project:
 - Copy the userService
 - Copy the <LoginSignup> component
 - Update the <AppHeader> component

Our app is built from the following components:

- TodoIndex (Routable, Smart component)
 - TodoList (Dumb)
 - TodoFilter (Dumb)
- TodoPreview (Dumb)
- TodoEdit (Routable, Smart)
- TodoDetails (Routable, Smart)

Terminology

- **Dumb (Presentational)** – A component that receives data and callbacks via props. It does not access the Redux store directly and does not interact with services.
- **Smart (Container)** – A component that dispatches actions to the Redux store, interacts with services, and can be routable (e.g., represent a page).

1. Store should manage the following state:
 - a. List of todos
 - b. isLoading
 - c. Current filterBy
 - d. User object
2. Confirm before deleting a todo
3. Add color to todo
4. Add filter by <select>: All | Active | Done
5. Show a progress-bar of todo's completion at the header and at the footer of the app

userService

Exports the following API:

```
export const userService = {  
  getLoggedInUser,  
  login,  
  logout,  
  signup,  
  getById,  
  query,  
  getEmptyCredentials  
}
```

The service manages a user entity:

```
const user = {  
  _id: "KAtTl",  
  username: "muki",  
  password: "muki1",  
  fullname: "Muki Ja",  
  createdAt: 1711490430252,  
  updatedAt: 1711490430999  
}
```

Add balance and activities:

```
{
```



```
balance: 10000,  
activities: [{txt: 'Added a Todo', at: 1523873242735}]  
}
```

User Balance

When user is loggedin, show the user balance next to his name, at the header.

When the user completes a task, his balance increases by 10 (balance just grow bigger, we don't decrease it when toggling the isDone to false)

<UserDetails>

Add a page to the app: <UserDetails> (Routable, Smart)

If the user is looking at his own user-details – allow changing his fullname and set preferences.

The image shows a user profile form with a light orange background. At the top left, the word "Profile" is written in bold black text. Below it, there are three input fields: "Name:" followed by a text box containing "Your name", "Color:" followed by a black color picker, and "BG Color:" followed by a black color picker. To the right of these fields is a grey "Save" button.

- Add to user object: `prefs: {color: 'black', bgColor: 'white'}`
- Also, render the user's activities list, example:
 - 2 minutes ago: Added a Todo: 'Wash the dishes'
 - Couple of hours ago: Removed the Todo: 'Talk to grandma'

More features

- Show a loading indication when todos are being loaded
- Show no “no todos to show..”, when there aren’t any todos
- Show Success and Failure messages in the component
- Make it look nice
- Push it to github-pages



Bonus

- When filtering by txt, support debouncing
- Add sorting and paging in the todoService

PART 2

- Move the store-related async code into store actions
- Split your store to 2 modules: userStore and todoStore
- Complete the app, make it look good and responsive

Bonus

- Build a backend to your app
 - TIP: copy and modify the backend from the carProj / missBug)
- Push the app to a cloud hosting (render.com)